

6.S081: 介绍

Adam Belay
abelay@mit.
edu

6.S081 目标

- 了解操作系统的设计与实现
- 亲手构建系统软件的实践经验
- 将扩展一个简单的操作系统 (xv6)
- Will learn about how hardware works (Risc-V)

你在 6.S081 中将要做的几件事

1. "你将构建一个网络栈的驱动程序，通过真正的互联网发送数据包
2. 你将重设计一个内存分配器，使其能够在多个核心上扩展
3. 你将实现 fork，并通过一种称为 copy-on-write 的优化使其高效

操作系统的目的是什么？

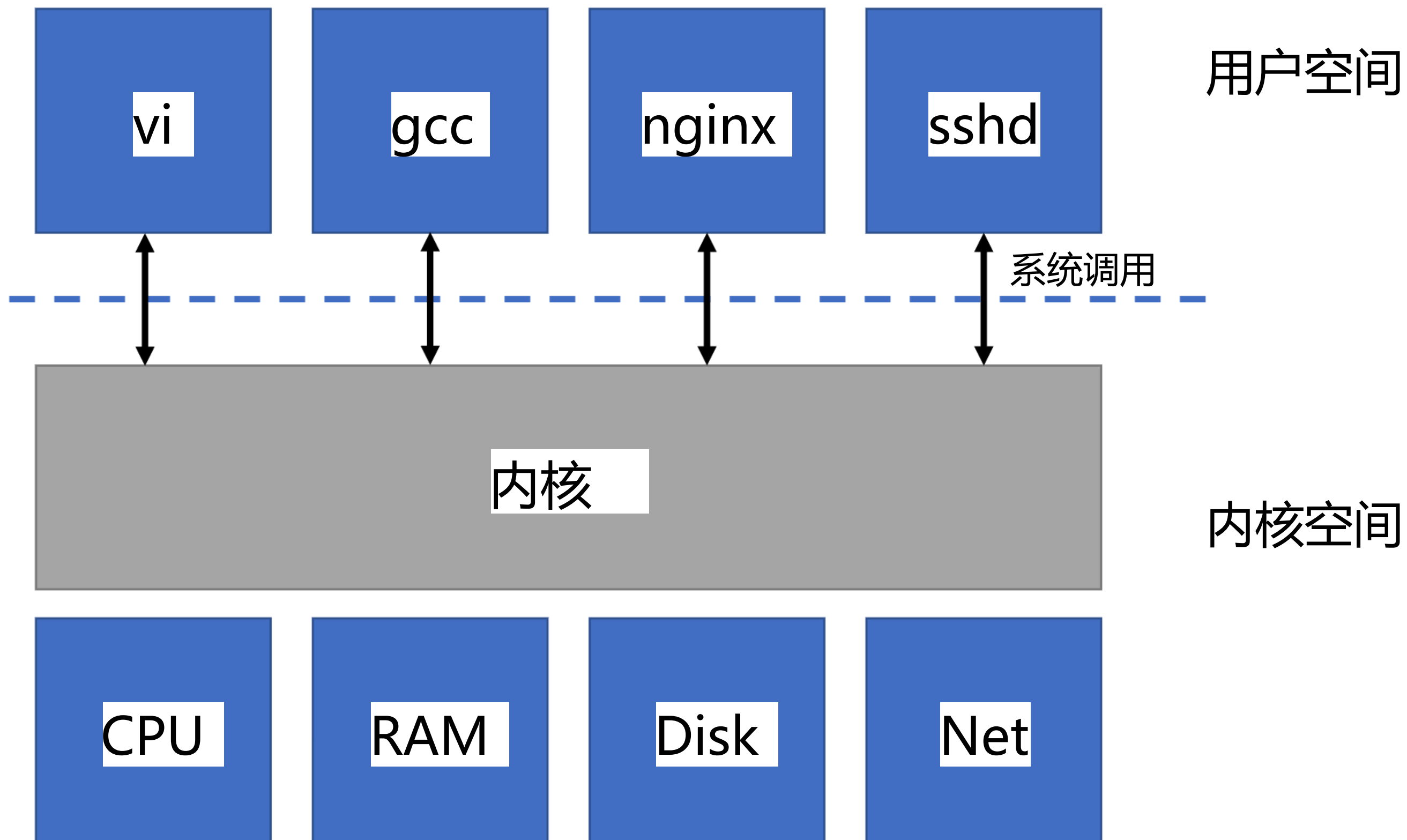
1. 抽象

- 隐藏硬件细节以实现移植性和便利性
- "不得妨碍高性能"
- 必须支持广泛的应用范围

2. 复用

- 允许多个应用程序共享硬件
- 隔离以包含错误并提供安全
- 共享以允许合作

OS 组织



OS 抽象

- 进程（正在运行的程序）
- 内存分配
- 文件描述符
- 文件名和目录
- 访问控制和配额
- 以及其他内容：用户、IPC、网络套接字、时间等

用户 <-> 内核接口

- 主要是系统调用
- 示例：

```
fd = open( "out" ,  
1); len = write(fd,  
    "hello\n" , 6); pid =  
fork();
```
- 看起来像函数调用，但其实不是

为什么操作系统很有趣

- 但它们并不容易构建：调试很难，一个错误可能会导致整个机器崩溃
- 设计权衡：
 - 效率 vs. 可移植性/通用性
 - 强大 vs. 简单
 - 灵活 vs. 安全
- 挑战：良好的正交性，特征交互
- 从智能灯泡到超级计算机的各种用途
- 演进的硬件：NVRAM, 多核, 200Gbit 网络

如果你想要学习：

- 200Gbit 网络 想要从工程角度真正理解计算机是如何工作的
- 想要构建未来的系统基础设施
- 想要解决 bug 和安全问题
- 关注性能

物流

在线资源

- 课程网站
 - <https://pdos.csail.mit.edu/6.S081/>
 - 课程时间表、课程政策、实验作业等
 - 2020 次课程的视频和笔记
- Piazza
 - <https://piazza.com/mit/fall2021/6s081>
 - 公告和讨论
 - 关于实验室和讲座提问

课程

1. 操作系统概念
2. xv6 案例研究 --- 一个简单的小型操作系统
3. 实验室背景和解决方案
4. 操作系统论文

- 每节课前提交一个问题
- 资源: xv6 书

Labs

- Goal: 实践经验
- 三种类型的实验室：
 1. 系统编程：下周截止
 2. OS 原语：例如，线程调度
 3. 操作系统扩展：例如，网络驱动程序

合作

- 课堂上或 Piazza 上可以自由提问和讨论实验作业
- 讨论很好
 - 但所有解决方案（代码和书面工作）必须是您自己的
 - 承认他人的想法（例如，同学、开源软件、stackoverflow 等）
- 不要发布您的解决方案（包括在 github 上）

Covid-19 和面对面学习

- 必须佩戴口罩；需正确佩戴
- 如果您有症状或检测呈阳性...
 - 不要上课，立即联系我们
 - 我们将与您合作提供课程材料

评分

- 70% 实验室作业，基于您将要运行的相同测试
- 20% 实验检查会议
 - 我们将询问随机选择的实验室问题以在办公室时间进行讨论
- 10% 作业和课堂/皮亚扎参与

返回系统调用

- 我将展示如何使用系统调用示例
- Will use xv6, 你将在实验室中使用的同一操作系统
- xv6 类似于 UNIX 或 Linux, 但要简单得多
 - 为什么? 这样你可以理解整个东西。
- 为什么是 UNIX?
 - 简洁的设计, 广泛使用: Linux, OSX, Windows (主要是)
- xv6 在 Risc-V 上运行, 就像 6.004 一样
- 你将使用 Qemu 运行 xv6 (模拟)

System call

Description

<code>int fork()</code>	Create a process, return child's PID.
<code>int exit(int status)</code>	Terminate the current process; status reported to <code>wait()</code> . No return.
<code>int wait(int *status)</code>	Wait for a child to exit; exit status in <code>*status</code> ; returns child PID.
<code>int kill(int pid)</code>	Terminate process PID. Returns 0, or -1 for error.
<code>int getpid()</code>	Return the current process's PID.
<code>int sleep(int n)</code>	Pause for n clock ticks.
<code>int exec(char *file, char *argv[])</code>	Load a file and execute it with arguments; only returns if error.
<code>char *sbrk(int n)</code>	Grow process's memory by n bytes. Returns start of new memory.
<code>int open(char *file, int flags)</code>	Open a file; flags indicate read/write; returns an fd (file descriptor).
<code>int write(int fd, char *buf, int n)</code>	Write n bytes from buf to file descriptor fd; returns n.
<code>int read(int fd, char *buf, int n)</code>	Read n bytes into buf; returns number read; or 0 if end of file.
<code>int close(int fd)</code>	Release open file fd.
<code>int dup(int fd)</code>	Return a new file descriptor referring to the same file as fd.
<code>int pipe(int p[])</code>	Create a pipe, put read/write file descriptors in <code>p[0]</code> and <code>p[1]</code> .
<code>int chdir(char *dir)</code>	Change the current directory.
<code>int mkdir(char *dir)</code>	Create a new directory.
<code>int mknod(char *file, int, int)</code>	Create a device file.
<code>int fstat(int fd, struct stat *st)</code>	Place info about an open file into <code>*st</code> .
<code>int stat(char *file, struct stat *st)</code>	Place info about a named file into <code>*st</code> .
<code>int link(char *file1, char *file2)</code>	Create another name (file2) for the file file1.
<code>int unlink(char *file)</code>	Remove a file.